

CODYSSEY AI/SW BASIC | DATA STRUCTURES & ALGORITHMS

Mini Git

CLI 기반 커밋 그래프 엔진

외장 그래프 패키지 & 표준 정렬 API 없이 순수 Python만으로 구축한 분산 버전 관리 시스템(DVCS) 코어 아키텍처



발표자: 최성민



8/8 Tests 100% PASS



Pure Python 3.10+

과제 핵심 목표 및 엄격한 설계 제약사항

🎯 미션 핵심 목표

- ▶ DVCS 코어 체득: Git의 커밋 그래프 및 해시 무결성 원리를 밑바닥부터 구현
- ▶ DAG 모델링: 커밋을 방향성 비순환 그래프로 모델링하고 브랜치 포인터 관리
- ▶ 위상 정렬 LOG: 부모 커밋이 자식보다 항상 먼저 출력되는 인과율 보장 로그
- ▶ 무방향 최단 경로: 브랜치 간 최단 경로 탐색 및 사전순 타이브레이크 보장
- ▶ 역색인 검색 엔진: 단일 룩업 O(1) 키워드 및 작성자 검색 파이프라인 구축

🚫 엄격한 라이브러리 제약조건

- ▶ 그래프 패키지 금지: NetworkX, igraph 등 외장 그래프 라이브러리 사용 전면 배제
- ▶ 표준 정렬 API 금지: sorted(), list.sort() 등 Python 내장 정렬 API 일체 사용 금지
- ▶ 순수 알고리즘 구현: 인접 리스트, BFS 큐, Kahn 위상 정렬, 병합 정렬 직접 코딩
- ▶ 인메모리 경량화: 네트워크 및 디스크 I/O 없이 메모리 상의 해시맵 상태로 운영
- ▶ 보안 및 표준 준수: shlex 토큰화, 대소문자 무관 커맨드, 표준 에러 체계 확립

단일 책임 원칙(SRP) 기반 계층 아키텍처

 Presentation (CLI)

- ▶ cli.py: 대화형 REPL 루프 및 입출력 정규화
- ▶ shlex.split: 큰따옴표 포함 문자열 공백 안전 파싱
- ▶ dispatch(): 명령어 대소문자 무관 라우팅
- ▶ 에러 표준화: Invalid args, Unknown branch/commit

 State Engine (Domain)

- ▶ repo.py: 저장소 형상 상태 단일 진실 공급원
- ▶ branches: 브랜치명 -> 최신 해시 가변 포인터 맵
- ▶ current_branch: HEAD 심볼릭 체크아웃 브랜치 관리
- ▶ Commit Node: 6자리 해시, 부모 목록, 타임스탬프, order

 Algorithms Engine

- ▶ graph.py: 위상 정렬, BFS 최단 경로, DFS 조상 탐색
- ▶ sorting.py: merge_sort (O(N log N) 안정 병합 정렬)
- ▶ index.py: InvertedIndex (토큰 및 작성자 역색인)
- ▶ 무상태성: 순수 데이터 디렉터리만 인자로 받아 재사용

커바 그래프 자료구조 — 왜 DAG인가?

 시간의 인과율과 비순환성 보장

- ▶ 방향성 (Directed): 간선은 항상 '자식 노드 -> 부모 노드' 방향으로 연결
- ▶ 인과성 (Causality): 새 커바 생성 시 부모는 이미 확정된 과거 노드만 지정 가능
- ▶ 미래 참조 불가: 미래 커바은 아직 해시가 없으므로 참조 불가 -> 사이클 원천 차단
- ▶ Commit.parents: 0개(루트), 1개(일반), 2개 이상(Merge) 부모 리스트 관리
- ▶ 단일 해시맵 O(1): Repository.commits[hash]로 임의 커바 상수 시간 접근

 사이클 발생 시 시스템 파멸적 영향

- ▶ 무한 루프 발생: 조상 탐색(DFS) 및 최단 경로(BFS) 순회 시 탈출 조건 붕괴
- ▶ 위상 정렬 불가능: 진입 차수가 0으로 떨어지지 않아 선후 관계 정의 불가
- ▶ 머지 베이스 붕괴: 공통 조상(LCA)을 특정할 수 없어 3-way 병합 불능 상태 초래
- ▶ 데이터 무결성: 해시 체인이 상호 참조를 일으켜 암호학적 머클 트리 성질 파괴

부모 커밋 선행 출력 — Kahn 알고리즘 위상 정렬

Kahn 알고리즘 3단계 절차

- ▶ 1단계: 진입차수 산출: 부모 -> 자식 간선에 대해 자식 노드의 indegree 계산
- ▶ 2단계: 큐 시딩: 부모가 없는 루트 커밋(`indegree == 0`)들을 큐에 등록
- ▶ 결정론적 순서: 동률 시 커밋 생성 순번(`order`) 기준 안정 정렬하여 시딩
- ▶ 3단계: 큐 소비 & 삭감: 큐에서 노드 `u` 추출 후 자식 `v`의 `indegree`를 1 감소
- ▶ 자식 큐 투입: `indegree[v]`가 0이 되는 순간(모든 부모 출력 완료) 큐에 추가

```
ready = [c.hash for c in by_creation if indegree[c.hash] == 0]
```

복잡도 분석 & 엔지니어링 의의

- ▶ 시간 복잡도: $O(V + E) + O(V \log V)$ — 정점 및 간선 1회 순회로 선형 수행
- ▶ 공간 복잡도: $O(V + E)$ — 인접 리스트 및 indegree 테이블 유지
- ▶ 단순 나열과의 차이: 자식이 부모보다 먼저 보이는 일반 최신순 로그의 한계 극복
- ▶ 외부 이력 대비: 생성 순서에 의존하지 않고 진짜 indegree를 추적하여 Replay에도 안전

최단 경로 탐색 — 무방향 간선 모델링 & BFS

왜 간선을 무방향으로 간주하는가?

- ▶ 형제 브랜치 분기: feature와 main 브랜치 커밋은 공통 조상에서 갈라진 형제 관계
- ▶ 단방향 탐색의 한계: 자식 -> 부모만 허용 시 공통 조상 도달 후 다른 브랜치로 역주행 불가
- ▶ 양방향 간선 확장: adjacency[p].append(c) & adjacency[c].append(p)로 그래프 구성
- ▶ 브랜치 횡단 경로: login -> root -> payment 형태의 브랜치 간 실질 거리 측정 가능

BFS 거리 맵 계산 메커니즘

- ▶ 비가중치 최단 경로: 모든 간선 가중치=1이므로 다익스트라 대신 BFS가 가장 최적
- ▶ 타깃 기점 탐색: 목적지(end)로부터 도달 가능한 모든 노드의 최단 홉 거리(dist_to_end) 계산
- ▶ 도달 불가 검증: start 노드가 dist_to_end 맵에 없으면 즉시 'No path' 반환
- ▶ 시간/공간 복잡도: $O(V + E)$ 선형 시간에 그래프 전체 최단 거리 맵 확보

사전순 최단 경로 선택 — 그리디 역추적 수학적 증명

💎 다이아몬드 머지 경로 경쟁

- ▶ 동일 거리 경쟁: $a0 \rightarrow \{b1, c1\} \rightarrow d2$ 구조에서 $a0 \rightarrow b1 \rightarrow d2$ 와 $a0 \rightarrow c1 \rightarrow d2$ 는 둘 다 2홉
- ▶ 요구 규격: 경로를 'h1->h2->...' 결합 시 사전순(Lexicographical) 최솟값 선택
- ▶ 단순 접근의 비효율: 모든 최단 경로를 백트래킹으로 전수 열거 시 $O(K!)$ 폭발 위험
- ▶ 테스트 검증: 'b1' < 'c1'이므로 $a0 \rightarrow b1 \rightarrow d2$ 가 선택됨을 완벽 검증

▶ 그리디 역추적 정당성 증명

- ▶ 길이의 동일성: 모든 최단 경로는 홉 수(문자열 블록 수)가 D로 완전히 동일함
- ▶ 사전순 정의: 같은 길이 문자열은 맨 처음 달라지는 원소 크기에 의해 순서 결정
- ▶ 탐욕적 선택 속성: 매 걸음마다 dist가 1 줄어드는 이웃 중 해시가 가장 작은 노드 선택
- ▶ 최적 부분 구조: 국소 최적 선택(Local Optimum)이 전체 사전순 최소(Global)와 일치
- ▶ 복잡도 혁신: 전수 열거 없이 $O(V + E)$ 내에 최종 최단 경로를 직접 재구성

조상 커밋 추적 알고리즘 — DFS 순회 & 중복 방지

스택 기반 깊이 우선 탐색 (DFS)

- ▶ 단방향 부모 추적: 시작 노드의 parents 리스트부터 스택에 투입하여 재귀 추적
- ▶ 자기 자신 제외: 시작 커밋 해시는 결과 집합에 포함하지 않고 순수 조상만 수집
- ▶ 미존재 커밋 방어: 저장소에 없는 해시 입력 시 None 반환 및 CLI에서 에러 처리
- ▶ 루트 커밋 질의: 부모가 없는 루트 커밋 질의 시 공집합(set()) 정확히 반환

다이아몬드 머지 중복 방문 차단

- ▶ 공통 조상 문제: 복수 브랜치가 병합된 경우 동일 조상이 여러 부모 경로를 통해 중복 노출
- ▶ seen 집합 활용: 해시셋(seen: set[str])을 통해 이미 방문한 노드는 스택 확장 생략
- ▶ 시간 복잡도: $O(V_{ancestors} + E_{ancestors})$ — 도달 가능한 부분 그래프 크기에만 비례
- ▶ 공간 복잡도: $O(V_{ancestors})$ — 방문 해시셋 및 DFS 호출 스택 크기

역색인 검색 엔진 — O(1) 초고속 조회 아키텍처

📖 역색인(Inverted Index) 구축 원리

- ▶ 사전 색인화 (Pre-indexing): 커밋 생성 (Write-time) 시점에 텍스트를 파싱하여 색인에 등록
- ▶ 토큰화 & 정규화: `message.lower().split()`으로 공백 분리 후 키워드 테이블에 추가
- ▶ 2종 인덱스 지원: `by_keyword[token] -> [hashes]`, `by_author[name] -> [hashes]`
- ▶ 불변 리스트 반환: 외부 수정 오염 방지를 위해 조회 시 리스트 복사본 반환

⚡ 순회 검색 vs 역색인 성능 비교

- ▶ 전체 순회 검색: 매 SEARCH 호출마다 전체 커밋 N개 메시지를 전수 스캔: $O(N \times L)$
- ▶ 역색인 검색: 단 1회의 해시 테이블 룩업으로 매칭 해시 추출: 평균 $O(1) + O(K)$
- ▶ 대규모 환경 격차: 커밋 10만 건 환경에서 순회 검색 대비 수천 배 이상의 속도 향상
- ▶ 메모리-시간 트레이드오프: 약간의 인덱스 오버헤드로 읽기 성능을 극대화하는 표준 IR 기법

자체 구현 정렬 엔진 — 병합 정렬과 안정성 증명

분할 정복 병합 정렬 메커니즘

- ▶ 제약 준수: `sorted()`, `list.sort()`를 일체 배제하고 `merge_sort()` 직접 구현
- ▶ 재귀 분할 (Divide): 배열을 중간 지점(`mid = len // 2`)에서 균등하게 절반 분할
- ▶ 결합 (Conquer & Merge): 두 정렬 리스트의 선두 원소를 투 포인터(`i, j`)로 비교 병합
- ▶ 다양한 키 지원: `key=lambda c: c.timestamp`, `key=lambda c: c.author` 유연한 정렬 지원

복잡도 및 안정 정렬(Stability) 보장

- ▶ 시간 복잡도 증명: $T(N) = 2T(N/2) + O(N) = O(N \log N)$ (최선/평균/최악 동일)
- ▶ 퀵 정렬 대비 우위: 퀵 정렬의 피벗 불균형에 따른 최악 $O(N^2)$ 성능 퇴화 위험 원천 제거
- ▶ 안정 정렬(Stable) 원리: `key(left[i]) <= key(right[j])` 비교 시 등호(`<=`)로 좌측 우선 채택
- ▶ 실무적 의미: 동일 작성자 정렬 시 최초 커밋 생성 순번(`order`)이 100% 보존됨

시스템 신뢰성 & 보안 엔지니어링 (Hardening)

 **6자리 SHA-1 해시 유일성 파이프라인**

- ▶ 해시 생성 포맷: `raw = f'{order}:{salt}:{message}:{timestamp}'` 바이트 스트림 생성
- ▶ 생일 역설 공간 분석: $16^6 = 16,777,216$ 공간에서 $k \approx 4,818$ 건 시 50% 확률로 충돌 가능
- ▶ 솔트(Salt) 재시도 루프: 충돌 감지 시 salt를 1 증가시켜 즉각 유일 해시 재발급
- ▶ 테스트 재현성: 단조 카운터 기반으로 일관된 결정론적 테스트 재현성 보장

 **애플리케이션 레벨 보안 가드레일**

- ▶ 인젝션 원천 차단: `shlex.split()` 렉서 파싱 및 `os.system` 배제로 셸 메타문자 무력화
- ▶ ReDoS 및 OOM 방어: 정규식 대신 `split()` 사용, 대용량 토큰 인덱싱 상한선 설계
- ▶ 민감정보 파라미터 은닉: 로그 출력 시 API Key, 토큰 마스킹(`hide_parameters`) 아키텍처 제시
- ▶ 도메인 예외 캡슐화: 비정상 조작 시 시스템 크래시 없이 표준 `RepoError` 메시지 반환

대규모 확장성 로드맵 & 최종 평가 결론

🚀 10만 건 커밋 대규모 확장 아키텍처

- ▶ 인접 리스트 상시 캐싱: 질의 시마다 즉석 생성하던 인접 리스트를 커밋 시점에 증분 캐싱
- ▶ 양방향 BFS 도입: 출발점과 도착점 동시 탐색으로 공간 복잡도 $O(b^d)$ -> $O(2 \cdot b^{(d/2)})$ 축소
- ▶ Commit-Graph 영속화: 실제 Git 2.18+ 규격처럼 바이너리 오프셋 테이블로 디스크 I/O 최적화
- ▶ 3-way Merge & Diff: LCS 동적 계획법 Myers Diff 및 공통 조상(LCA) 기반 병합 확장 완비

🏆 최종 검증 결과 요약

- ▶ 단위/통합 테스트: test_mini_git.py 8개 테스트 스위트 100% PASS
- ▶ 규칙 동기화 검증: GEMINI.md <--> AGENTS.md 6개 디렉터리 100% 동기화 PASS
- ▶ 다이어그램 문법: 36개 마크다운 내 Mermaid 다이어그램 100% PASS
- ▶ 구술 평가 판정: PASS (경험정도: 능숙한 재구현 가능 / Level 5 확정)

